

GenAI Programs used:

Program 1: ChatGPT, chatgpt.com (accessed through Website)

Program 2: Claude, claude.ai (Accessed through website interface)

Prompt used:

Objective

Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess.
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output

text

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

ChatGPT code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int secretNumber = 7;
```

```
    int guess;
```

```
    int attempt = 1;
```

```
    int won = 0;
```

```
    printf("Guess a number between 1 and 10.\n");
```

```
    while (attempt <= 3) {
```

```
        printf("Attempt %d/3. Enter your guess: ", attempt);
```

```
        scanf("%d", &guess);
```

```
        if (guess == secretNumber) {
```

```

        printf("Correct! You win!\n");
        won = 1;
        break;
    } else if (guess < secretNumber) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }

    attempt++;
}

if (!won) {
    printf("You lose! The secret number was %d.\n", secretNumber);
}

return 0;
}

```

Claude Code:

```

/*
 * guess_number.c
 *
 * Objective:
 * The program selects a fixed secret number between 1 and 10.
 * The user has up to 3 attempts to guess it correctly.
 * After each guess, the program tells the user whether their
 * guess was too high, too low, or correct.
 *
 * Approach:
 * - The secret number is hard-coded (no randomization required
 *   by the assignment spec).
 * - A for-loop bounds the number of attempts to 3.
 * - printf/scanf handle prompting and input.
 * - if-else statements compare the guess to the secret number.
 * - A flag variable (guessedCorrectly) tracks whether the user
 *   won, so we can break out of the loop immediately on a
 *   correct guess and still know which final message to print
 *   after the loop ends.
 */

#include <stdio.h>

```

```

int main(void)
{
    const int SECRET_NUMBER = 7; /* Fixed secret number (1-10) */
    const int MAX_ATTEMPTS = 3; /* User gets up to 3 tries */

    int guess; /* Stores the user's current guess */
    int guessedCorrectly = 0; /* 0 = not yet correct, 1 = correct */

    printf("Guess a number between 1 and 10.\n");

    /* Loop runs at most MAX_ATTEMPTS times, but stops early (via
     * break) as soon as the user guesses correctly. */
    for (int attempt = 1; attempt <= MAX_ATTEMPTS; attempt++)
    {
        printf("Attempt %d/%d. Enter your guess: ", attempt, MAX_ATTEMPTS);
        scanf("%d", &guess);

        if (guess == SECRET_NUMBER)
        {
            /* Correct guess: record the win and exit the loop now,
             * instead of waiting for remaining attempts. */
            printf("Correct! You win!\n");
            guessedCorrectly = 1;
            break;
        }
        else if (guess < SECRET_NUMBER)
        {
            printf("Too low! Try again.\n");
        }
        else /* guess > SECRET_NUMBER */
        {
            printf("Too high! Try again.\n");
        }
    }

    /* If the loop finished without a correct guess, the user has
     * used all attempts and loses. */
    if (!guessedCorrectly)
    {
        printf("Sorry, you lose! The correct number was %d.\n", SECRET_NUMBER);
    }

    return 0;
}

```

e. Analysis

Correctness: Both programs do everything the assignment asks. They both pick a fixed secret number, let the user guess up to 3 times, tell them if they're too high, too low, or right, and stop early if they guess correctly. Neither one has bugs or missing parts.

Execution Time: Both programs do basically the same amount of work each time through the loop — one comparison and one print statement — and both stop after at most 3 tries. Claude's version uses a for loop and ChatGPT's version uses a while loop, but that doesn't actually make one faster than the other here. They run in the same amount of time.

Space Complexity: Both programs only use a few simple variables (the secret number, the guess, a counter, and a flag to check if the user won). Neither one uses extra memory like arrays or dynamic allocation, so they use about the same amount of memory.

Maintainability: This is the main place they're different. Claude's version uses named constants like `SECRET_NUMBER` and `MAX_ATTEMPTS` instead of just typing the numbers 7 and 3 directly into the code. It also has comments explaining what the program does and why certain things were done a certain way. ChatGPT's version doesn't have any comments and just uses the plain numbers 7 and 3 straight in the code. That means if you wanted to change something later, like making it 5 attempts instead of 3, Claude's version only needs one small change, but ChatGPT's version means digging through the code to find and change the number everywhere it shows up. Claude's version is just easier to read and change.

f. Selection and Justification

I picked Claude's version to build on. Both programs work the same and run about the same speed, so those weren't the deciding factors. What made the difference was maintainability — Claude's version is easier to read and easier to update later because it uses named constants and has comments explaining what's going on. Since everything else was basically tied, that's why I went with Claude's version.

G. Changes/addons

After picking Claude's code I added Range Validation to prevent the user from entering any number outside of 1-10. And I also added the function to prevent any input that isn't a number so it doesn't error out. Both of these are looped so the user is given another chance to enter a valid input.